

MIPS Procedure Calling and Register Conventions

Overview of Procedure Calling

When one procedure (caller) calls another (callee), the following actions must be performed:

1. Caller places arguments in registers for callee to use.
2. Caller transfers control to callee.
3. Callee acquires storage for its data.
4. Callee performs its operations.
5. Callee places the result in registers for caller.
6. Callee returns control to caller.

Example in C

```
1 int foo(int i, int j) {  
2     int k = 1;  
3     ...  
4     m = bar(5, 7);  
5     ...  
6     return 3;  
7 }  
8  
9 int bar(int x, int y) {  
10     int z = 2;  
11     ...  
12     return 9;  
13 }
```

Register Usage Conventions

- **\$a0 - \$a3** (Registers 4–7): Arguments
- **\$v0, \$v1** (Registers 2–3): Return values
- **\$t0 - \$t9** (Registers 8–15, 24–25): Temporaries
- **\$s0 - \$s7** (Registers 16–23): Saved variables
- **\$gp** (Register 28): Global pointer
- **\$sp** (Register 29): Stack pointer
- **\$fp** (Register 30): Frame pointer
- **\$ra** (Register 31): Return address

Passing Arguments to Callee

- Arguments are placed in \$a0–\$a3.
- Caller may need to save its own arguments before the call.

Transferring Control

Use the `jal` (Jump and Link) instruction:

```
1 jal bar
```

- Stores return address in **\$ra**.
- If caller needs **\$ra**, it must save it before the call.

Acquiring Storage in Callee

- Data is stored in \$t and \$s registers.
- Problem: Caller and callee might both use the same registers.
- Solution: Caller saves **\$t***, Callee saves **\$s***.

Split Responsibilities:

- Caller: Saves any **\$t*** it uses before call and restores after return.
- Callee: Saves and restores any **\$s*** it uses.

Returning Result to Caller

- Callee stores result in `$v0` or `$v1`.
- Caller retrieves result from these registers.

Returning Control to Caller

<code>jr \$ra</code>

- `jr` jumps to the return address stored in `$ra`.

Summary: Procedure Call Steps

1. Caller places arguments in `$a0–$a3`.
2. Caller saves needed registers (e.g., `$ra`, `$t0–$t9`).
3. Caller uses `jal` to jump to callee.
4. Callee saves `$s*` registers if used.
5. Callee does work, places return value in `$v0`.
6. Callee uses `jr $ra` to return.
7. Caller restores saved registers.